16 Soubory a serializace – Ukládání a načítání dat, formáty souborů

Co je to soubor?

Soubor jako takový je jenom pojmenovaná skupinka nějakých dat. Data můžou být různého typu, obrazové, textové, zvukové, programové… Soubory se používají k organizaci dat a mohou být jakkoliv upravovány, čteny a posílány dál. Každý soubor má nějaké jméno, které následně slouží k identifikace a adresování, kde jsou v paměti data uložená.

Java pro práci se soubory využívá knihovnu java.IO – Input Output.

Čtení ze souboru je thread-safe, ale zápis thread-safe není a musí se používat například RWL – Read Write Lock Conccurent patterny.

Co je to serializace?

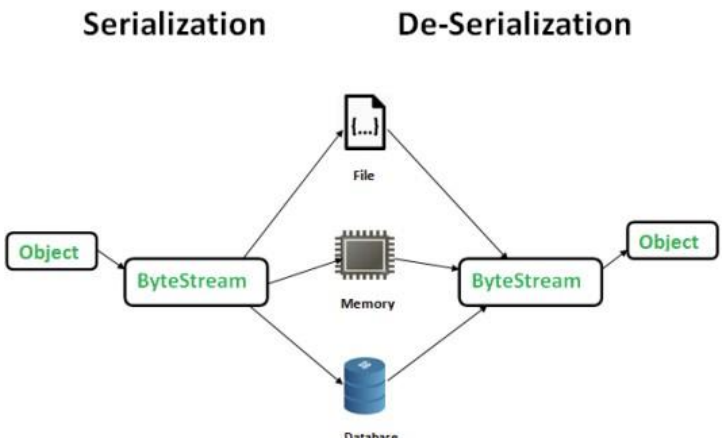
Serializace je nějaký proces, převod objektů a nebo datových struktur do formátu, který bude jednoduše přenositelný. Typicky se využívá například pro ukládání stavu aplikace, ukládání a načtení konfiguračních souborů a nebo obyčejný přenos dat mezi procesy v počítači. Během serializace jsou data přenášena jako série bytů, které se následně zase deserializují do původní podoby.

Data, která jsou totiž po vypnutí programu na stacku nebo na heapu, se ztratí. Serializace nám umožňuje tyto objekty nebo datové struktury zachovat.

Java jako taková pro serializaci používá mechanismus Java Serialization, který umožňuje ukládat objekty ja disk nebo je přenášet přes síť.

Je spoustu různých formátů pro serializaci, nejznámější z nich jsou například XML a nebo JSON.

Ukládání a načítání dat



Serializování objektů

Ukládání dat funguje poměrně jednoduše. V Javě, pokud chceme nějaký objekt uložit, implementujeme dotřídy java.io.Serializable. Pozor, je vždy serializován celý objekt. Můžeme to obejít tím, že proměnné dáme klíčové slovo ***„transient“.*** Tím říkáme, že se proměnná nemá serializovat. Pokud nás zajímá, jestli je něco serializovatelné nebo ne, podíváme se jenom do dokumentace. Pokud implementuje knihovnu java.io.Serializable, tak ano, jinak ne.

K serializování objektů se využívá ObjectOutputStream – Stream je proud dat v bytové podobě. Pomocí metody .writeObject se následně objekt serializuje.

Důležité je mít také FileOutputStream – ten totiž vytvoří Stream a ObjectOutputStream tento výstupní Stream zabalí tak, aby se vědělo, že se jedná o serializovaný objekt.

V Javě se standartně serializovaným objektům dává koncovka .ser

V tomto případě do souboru src/employee.ser uložíme binární reprezentaci objektu e, tedy zaměstnance. Při deserializaci vytvoříme novou instanci Employee a ta bude obsahovat všechny hodnoty, které měl náš Employee před deserializací.

Je důležité poznamenat, že podobně jako při serializaci, při deserializaci se musí zachovat stejný datový typ a pořadí prvků, jinak by mohlo dojít k chybě.

```
1  package maturita;
2
3  import java.io.FileOutputStream;
4  import java.io.IOException;
5  import java.io.ObjectOutputStream;
6
7  public class Employee {
8      public static void main(String[] args) {
9          Employee e = new Employee(); // You need to create an instance of Employee
10         e.name = "ali";
11         e.address = "neco";
12         e.SSN = 111111;
13         e.number = 101;
14
15         try {
16             FileOutputStream fileOut = new FileOutputStream("src\\employee.ser"); // Corrected file path
17             ObjectOutputStream out = new ObjectOutputStream(fileOut); // Corrected class name
18             out.writeObject(e);
19             out.close();
20             fileOut.close();
21             System.out.println("Data jsou serializovana v src\\\\employee.ser"); // Corrected printf to println
22         } catch(IOException i) {
23             i.printStackTrace();
24         }
25     }
26
27     // Declare instance variables
28     private String name;
29     private String address;
30     private int SSN;
31     private int number;
```

Deserializace

V případě deserializace my načítáme postupně binární informace o objektu pomocí FileInputStream a ObjectInputStream. V tomto případě my využijeme metodu readObject, který má ObjectInputStream a přetypujeme informace z této metody do našeho Zaměstnance.

Důležité je samozřejmě mít vše ošetřené v try – catch blocku, který kontroluje případné problémy se čtením souboru, jako například ty, že soubor neexistuje, nebo to, že není možné vytvořit zaměstnance, protože třída neexistuje.

Pokud také je nějaké proměnná, kterou jsme neserializovali kvůli klíčovému slovu „transient“, nastaví se na defaultní hodnotu. V případě nějaékho čísla se bude jednat o nulu, v případě booleanu zase false.

```java
package maturita;
import java.io.FileInputStream;
import java.io.IOException;
import java.io.ObjectInputStream;

public class Employee {
    public static void main(String[] args) {
        Employee e = null;
        try {
            FileInputStream fileIn = new FileInputStream("/tmp/employee.ser");
            ObjectInputStream in = new ObjectInputStream(fileIn);
            e = (Employee) in.readObject(); // Corrected method name to readObject()
            in.close();
            fileIn.close();
        } catch (IOException i) {
            i.printStackTrace();
            return;
        } catch (ClassNotFoundException c) {
            System.out.println("Employee class not found");
            c.printStackTrace();
            return;
        }

        // Now you can use the 'e' object
        System.out.println("Deserialized Employee...");
        System.out.println("Name: " + e.name);
        System.out.println("Address: " + e.address);
        System.out.println("SSN: " + e.SSN);
        System.out.println("Number: " + e.number);
    }

    // Declare instance variables
    private String name;
    private String address;
    private int SSN;
    private int number;

    // Constructor
    public Employee(String name, String address, int SSN, int number) {
        this.name = name;
        this.address = address;
        this.SSN = SSN;
        this.number = number;
    }
}
```

Formáty souborů

Formát souboru nám říká, co můžeme v souboru najít. Je jich poměrně hodně a každý je rozdílný. Všechny soubory jsou ve skutečnosti binárními soubory, protože počítač je ukládá jako posloupnost bitů.

Textové soubory

Textové soubory jsou takové soubory, které v sobě mají text, kterému můžeme člověk při otevření rozumnět. Pokud bychom otevřeli binární soubor, nebudeme vědět, na co se koukáme. Příklad textového souboru může být právě třeba Word, textový soubor, nějaký kód nebo konfigurační soubory. Jde o to, že pokud obsah souboru přečteme, dostaneme třeba zpátky třeba „Kočičkyy“

Binární soubory

Binární soubory jsou takové soubory, které uchovávají binární informace. Nejsou čitelné lidským okem a jsou skládány do nějaékho formátu pro účely kompilace nebo interpretace. Vyznačují se také tím, že po přečtení souboru dostane přesně ty samá data, jako se nachází v souboru a nejsou žádné konverze, jako u textových, kde se následně bajty dekódují.

V případě binárních souborů, pokud bychom je přečetli, dostaneme změť náhodných znaků, nebo náhodných bajtů dat, třeba

Příkladem binárního souboru může být právě serializovaný soubor.

Čtení ze souboru ve více vláknovém prostředí

- **Čtení** ze souborů je možné (**thread-safe)**
- Pokud chceme ale **zapisovat**, musíme použít **Lock**, či thread-safe kolekci, která by měla jen jedno vlákno.

Funkce

- **Open**(<”*path-soubor.txt*”>,<”*r/w/rb…*”>) - **otevře soubor** s nastavením r/w/a, pokud neuspěje, vyhodí **výjimku**, která se chytá přes **OSError** (kde můžeme odchytávat, soubor nenalezen, nedostatečná práva …)

Metody

- Write()
- Read()
- Flush() - pro zapsání dat z bufferu (+vymazání) bufferu, kde jsou data k zápisu dočasně uchována
- Close() - po každém otevření, musí být soubor i **zavřen** (na to se např. používá v catch finally)
  *Př: Python má možnost otevřít soubor přes **with** open(), který soubor sám zavře*